Data Structures and Algorithms -3 (25CS2103E)

Recommendation System using DSA

Team Members
Salman Ahmed: 2520089070
P Venkata Harshavardhan: 2520089071

Under the Esteemed Guidance of
Dr Anitha Chekuri
Data Structures And Algorithms

Contents

- Abstract
- Introduction
- Literature survey
- Problem Statement and Objectives
- Hardware and Software requirements
- Results
- Conclusion
- Future Scope

Abstract

Recommendation system built from core DSA

User–item interactions (ratings, clicks, purchases) are stored in hash maps and an inverted index, and modelled as a bipartite graph.

Cosine similarity finds similar users and items. A max-heap retrieves the Top-K neighbours and the Top-N recommendations without sorting everything.

Collaborative and content-based filtering are combined and demonstrated on a small movie example.

O(1)

hash-map lookups of user and item data

Top-K

neighbours and picks from a max-heap

2 methods

collaborative + content-based filtering

Introduction

What

An algorithm that analyses preferences, behaviour and item data to predict the content most relevant to each user.

Why DSA

Hash maps give O(1) lookups, graphs model user–item links, heaps pick Top-N, and sorting and search rank the results.

How

Collect ratings, clicks and purchases, score similarity, rank the candidates and show the Top-K results.

Content-based filtering

Item feature vectors and cosine similarity match items to a user’s preferences.

Collaborative filtering

Nearest-neighbour search over the user–item matrix finds users with similar taste.

Literature Survey

GroupLens (1994)
Rating-based collaborative filtering of Usenet news using similar readers.

Amazon item-to-item (2003)
Precomputed similar-item table makes recommendations a fast lookup at scale.

Item-based CF (2001)
Compares items with cosine and correlation; scales better than user-based.

Matrix factorization (2009)
Latent factors for users and movies, built for the Netflix Prize data.

MovieLens datasets (2015)
Widely used rating datasets for testing recommendation algorithms.

Recommender survey (2005)
Content-based, collaborative and hybrid methods, plus cold-start and sparsity limits.

Problem Statement and Objectives

Problem Statement

Design an efficient, scalable recommendation system that suggests relevant items to every user using core data structures and algorithms.

Description

User–item data is large and sparse, so scanning and sorting all of it for each request is slow. Fast lookup, similarity search and Top-K ranking are needed.

Objectives

1. Represent interactions with hash maps, an inverted index and a bipartite graph.
2. Measure user and item similarity with cosine similarity.
3. Retrieve Top-K neighbours and Top-N picks with a max-heap.
4. Combine collaborative and content-based filtering in a movie example.
5. Analyse complexity, benefits and challenges.

Flow Chart

1. User–item data
ratings, clicks, purchases

2. Hash maps + inverted index
item → users who rated it

3. Candidate users
share at least one item

4. Cosine similarity
user vs user

5. Top-K neighbours
max-heap

6. Score unseen items
similarity-weighted ratings

7. Top-N recommendations
max-heap extraction

Steps 1–3 represent and narrow the data; steps 4–7 score, rank and deliver the results.

Results

```
Nearest neighbours of Asha:
  Bala      similarity = 0.839
  Farhan    similarity = 0.662
  Chitra    similarity = 0.325

Top recommendations for Asha:
  The Notebook predicted rating = 5.00
  Iron Man     predicted rating = 4.00
  Avengers     predicted rating = 3.56

Movies similar to 'Inception':
  Avengers     score = 1.000
  Iron Man     score = 1.000
  The Matrix   score = 1.000
```

Nearest neighbours

Bala (0.839) and Farhan (0.662) rate the same movies highly as Asha.

Top picks

Unseen movies are scored by similarity-weighted neighbour ratings.

Content-based

Genre vectors link Inception to Avengers, Iron Man and The Matrix.

Complexity: hash lookup O(1) | heapify O(n) | each heap extraction O(log n) | per query about O(C·d)

Hardware and Software requirements

Hardware

- Processor: Intel Core i3 / AMD Ryzen 3 or higher
- RAM: 4 GB minimum (8 GB recommended)
- Storage: 500 MB free disk space

Software

- Operating system: Windows 10/11 or Linux
- Language: Python 3.10 or later
- Editor: Visual Studio Code or PyCharm
- Libraries: Python standard library (heapq, collections, math)

Conclusion

Represent

Hash maps, an inverted index and a bipartite graph hold user–item data.

Compare

Cosine similarity measures how close users and items are.

Rank

A max-heap returns Top-K neighbours and Top-N picks in O(log n) per item.

Combine

Collaborative and content-based filtering work together in the movie example.

DSA gives recommendations that are fast, accurate, scalable and personalised.

Future Scope

Near term

- Load a real dataset such as MovieLens
- Add Pearson correlation and Euclidean distance
- Handle cold-start users with popular items

Medium term

- Matrix factorization
- Tries for autocomplete item search
- Cache similarity scores

Long term

- Real-time web service with a user interface
- Hybrid models that adapt to every interaction
- Diversity and fairness controls

Thank you

Recommendation System using DSA